

## 答疑课堂（一） | 第一章Rust基础篇思考题答案

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。

恭喜你学完前两章的内容了，基础篇和进阶篇一共有 20 讲，每一讲的内容都很重要，算是你入门 Rust 的重要基础，所以一定要多读几遍，争取学透。为了让你学思结合，我们在每节课的最后设计了对应的思考题，这节课我们就来处理这些问题。

我也看到很多同学在课程的后面回答了这些问题，此外还有一些其他的问题，提得也很精彩，所以我挑出一并放在这里，希望能为你解惑，对你有所启发，话不多说，我们马上开始吧！

做完思考题再来看答案会有收获。

### 基础篇

#### 🔗01 | 快速入门：Rust 中有哪些你不得不了解的基础语法？

### 思考题

Rust 中能否实现类似 JS 中的 number 这种通用的数字类型呢？

Rust 中能否实现 Python 中那种无限大小的数字类型呢？

## 答案

在 Rust 中，有多种数字类型，包括有符号和无符号整数、浮点数、复数等。和 JS 中的 number 类型相似，Rust 中的数字类型也支持基本的数学运算，例如加减乘除和取模等。不过，和 JS 的 number 不同，Rust 的数字类型都具有固定的位数，这意味着不同的数字类型有不同的取值范围。

此外，Rust 中的数值类型需要在编译时就确定它们的类型和大小，这些类型可以通过使用 Rust 内置的类型注解，或是灵活的小数点和后缀表示法来声明。而 crates.io 上有 num crate 可以用来表示通用的数字类型，具体是通过 trait 机制来实现的。

关于实现 Python 中无限大小的数字类型，Rust 不直接支持这个特性，但可以通过使用第三方库来实现。常用的第三方库包括 rug 和 num-bigint 等，它们提供了实现高精度计算的数据类型和函数，使 Rust 可以处理更大的整数和浮点数。这些库采用的是类似于 Python 的动态内存分配和存储机制，能够进行几乎无限大小的数字计算。

## 02 | 所有权（上）：Rust 如何管理程序中的资源？

### 思考题

1. 下面的示例将输出什么？

```
1 fn main() {
2     let s = "I am a superman.".to_string();
3
4     for i in 1..10 {
5         let tmp_s = s;
6         println!("s is {}", tmp_s);
7     }
8 }
```

[复制代码](#)

2. 一个由固定尺寸类型组成的结构体变量，如下面示例中的 Point 类型，在赋值给另一个变量时，采用的移动方式还是复制方式？

 复制代码

```
1 struct Point {  
2     x: i64,  
3     y: i64,  
4     z: i64  
5 }
```

## 答案

1. 无法通过编译，可以将第 5 行代码修改为：let tmp\_s = s.clone();。

修改后如下：

 复制代码

```
1 fn main() {  
2     let s = "I am a superman.".to_string();  
3  
4     for _ in 1..10 {  
5         let tmp_s = s.clone();  
6         println!("s is {}", tmp_s);  
7     }  
8 }
```

2. 由于 Point 没有实现 Copy trait，所以在赋值过程中会产生 Move。如果结构体实现了 Copy trait，则会进行复制而不是移动。

答案来自二夕 Thrower 和 Forest

## 🔗03 | 所有权（下）：Rust 中借用与引用的规则是怎样的？

## 思考题

1. 请思考，为何在不可变引用存在的情况下（只是读操作），原所有权变量也无法写入？

```
1 fn main() {  
2     let mut a: u32 = 10;  
3     let b = &a;  
4     a = 20;  
5  
6     println!("{}", b);  
7 }
```

2. 请回答，可变引用复制的时候，为什么不允许 copy，而是 move？

## 答案

1. 不可变引用的作用域跨越了所有权变量的写入过程，意味着同一个作用域同时存在可变引用和不可变引用，编译器为了防止读取错误，不能通过编译。可以把 `a = 20` 放到引用之前，即可编译通过。
2. 可变引用如果可以 Copy，就违反了可变引用不能同时存在的规则，因此只能 Move。

不可变借用，从字面理解就是借出去了就不能变了，所以既然保证不变了，那这样的借用当然可以被借出去 N 次（原变量不可变是只限于在借出去的变量的有效生命周期内）。可变借用就是借出去随时有被改变的可能，在同一生命周期内借出去多次，有不确定性的被改变的风险，尤其在多线程中，所以就只让你借出去一次，既然存在有不确定性的被修改的可能，那这个时期肯定就不会让你再有不可以变借用了（因为随时会改变了，不可变借用本身也就不成立了）。Rust 这个逻辑看似很繁杂，实则逻辑环环相扣清晰很符合常规。

答案来自 Andyling 和 Citroen

## 🔗 04 | 字符串：对号入座，字符串其实没那么可怕！

## 思考题

`chars` 函数是定义在 `str` 上的，为什么 `String` 类型能直接调用 `str` 上定义的方法？实际上 `str` 上所有的方法，`String` 都能调用，请问这是为什么呢？

## 答案

因此在 String 上实现了 Deref trait, target 为 str。

另外补充一些字符串相关知识点。

Rust 中 `char` 是用于存放 unicode 单个字符的类型（固定 4 个字节）。String 类型只能放在堆上，通过引用所有权的形式和变量绑定，它的存储方式不是简单的 char 数组，而是 utf8 编码的字节序列，所以单独取这个序列的某一段切片，不一定能解析出具体的字符（程序里的 `String[a..b]`，这里的 a 和 b 已经是经过特殊处理的保证截取的有效性）。

[复制代码](#)

```
1 fn main() {
2     let s = "abcdefghijk".to_string();
3     let a = &s[..5];
4
5     let s = "我爱中国".to_string();
6     let a = &s[..5];
7 }
8
9 Running `target/debug/playground`
10 thread 'main' panicked at src/main.rs:7:15:
byte index 5 is not a char boundary; it is inside '爱' (bytes 3..6) of `我爱中国`
```

如果能取得有效的序列片段那就是 str 类型，但是程序里凡是用到绑定 str 类型变量的地方，则必须都是引用形式存在的（&str），因为 str 是引用的原始片段的那段真实数据，而 &str 类型是一个 FatPointer，它包括引用目标的起始地址和长度，所以 str 和 &str 是完全两个不同的概念。

u8 就是一个存储 0 到 255 大小的类型，因为一个字节就是 8 位，所以[u8, N]可以看做是程序的任何类型数据的二进制表示形式。

答案来自 Citroen

## 05 | 复合类型（上）：结构体与面向对象特性

### 思考题

可以给 i8 类型做 impl 吗？

## 答案

基本数据类型无法实现 impl，不过我们可以通过 trait 给基本数据类型添加操作的方式来实现。

[复制代码](#)

```
1 trait Operate {  
2     fn plus(self) -> Self;  
3 }  
4  
5 impl Operate for i8 {  
6     fn plus(self) -> Self {  
7         self + self  
8     }  
9 }  
10  
11 fn main() {  
12     let a = 1i8;  
13     println!("{}", a.plus());  
14 }
```

另外可以用 newtype 模式对 `i8` 封装一下，再 impl。

答案来自下雨天和约书亚

## 🔗 06 | 复合类型（下）：枚举与模式匹配

### 思考题

match 表达式的各个分支中，如果有不同的返回类型的情况，应该如何处理？

## 答案

作为静态类型语言，match 返回的类型必须在编译期就被确定，也就意味着 match 必须返回相同的类型。在这个前提下，如果要返回不同的类型，那么切入点就只能是：“返回同一个类型，但是这个类型能表示（承载）不同的类型”，那就只能是本节课讲的枚举 enum 了。

[复制代码](#)

```

1  enum Number {
2      Int(i32),
3      Float(f64),
4      None
5  }
6
7
8  fn get_number(condition: i32) -> Number {
9      match condition {
10         1 => Number::Int(10),
11         2 => Number::Float(3.14),
12         _ => Number::None
13     }
14 }
15
16 fn main() {
17     let value = get_number(1);
18     match value {
19         Number::Int(i) => println!("int {}", i),
20         Number::Float(f) => println!("float {}", f),
21         Number::None => println!("not number"),
22     }
23
24     let value = get_number(2);
25     match value {
26         Number::Int(i) => println!("int {}", i),
27         Number::Float(f) => println!("float {}", f),
28         Number::None => println!("not number"),
29     }
30
31     let value = get_number(3);
32     match value {
33         Number::Int(i) => println!("int {}", i),
34         Number::Float(f) => println!("float {}", f),
35         Number::None => println!("not number"),
36     }
37 }

```

答案来自 -Hedon 🌈

## 🔗07 | 类型与类型参数：如何给 Rust 小助手提供更多信息？

### 思考题

如果你给某个泛型实现了一个方法，那么还能为一个具化类型再实现同样的方法吗？

## 答案

“为泛型实现了一个方法，能否再为具化类型实现一个同名方法”，取决于这个泛型能否表示相应的具化类型。比如为泛型 `T` 和 `String` 实现了相同的方法，由于 `T` 没有施加任何约束，它可以代表 `String`。那么当调用方法时，对于具化类型 `String` 来说，要调用哪一个呢？因此会出现歧义，编译器会报错：方法被重复定义了。

但如果给泛型 `T` 施加了一个 `Copy` 约束，要求 `T` 必须实现了 `Copy trait`，那么就不会报错了，因为此时 `T` 代表不了 `String`，所以调用方法不会出现歧义。但如果再为 `i32` 实现一个同名方法就会报错了，因为 `i32` 实现了 `Copy`，它可以被 `T` 表示。

答案来自古明地觉

## 🔗 08 | Option、Result、迭代器及实际类型中所有权问题

### 思考题

你可以用同样的思路去研究一下，看看如何拿到 `HashMap` 中值的所有权。

🔗 <https://doc.rust-lang.org/std/collections/struct.HashMap.html>

## 答案

`HashMap` 实现了 `into_iter()`，因此可以用 `for` 语句获取其所有权。

📄 复制代码

```
1 for (k, v) in myhash { //`myhash` moved due to this implicit call to `.into_iter(  
2     // todo:  
3     // 这里会获得v的所有权，并且消耗掉myhash  
4 }  
5  
6 println!("{:?}", myhash); //value borrowed here after move
```

答案来自 PEtFiSh 和 Ransang

## 🔗 09 | 初识 Trait：协议约束与能力配置



## 思考题

如果你学习或者了解过 Java、C++ 等面向对象语言的话，可以聊一聊 trait 的依赖和 OOP 继承的区别在哪里。

## 答案

trait 的依赖：小明要听从数学老师，语文老师，英语老师的话。老师之间是平等关系，多个依赖平等，最小依赖选择自己喜欢的功能。

OOP 继承：小明要听他爸、他爷爷、他曾祖父的话。继承之间存在父子关系，继承过来一堆破属性和方法，也许根本不是自己想要的，还要负重前行。

答案来自下雨天

## 🔗 10 | 再探 Trait: Trait + 类型 = Rust 的大脑

## 思考题

请谈谈在函数参数中传入 `&dyn TraitA` 与 `Box` 两种类型的区别。

## 答案

Rust 生命周期的独特设计，导致了该语言需要设计一些处理方式应对特殊情况，比如生命周期的标注（主要是给编译器进行代码处理时的提示）。事实上，我们在日常开发中应该避免一些陷入复杂情况的方式：比如，传入参数都用引用（borrow），传出结果都应该是 owner。Rust 也为我们提供了处理各种情况的工具。所以，一般来说，我们应该在传入参数的时候用 `&dyn T`，传出结果用 `Box<dyn T>`。

此外，`&dyn TraitA` 没有所有权，而 `Box<dyn TraitA>` 有所有权。`&dyn TraitA` 是借用，`Box` 会转移所有权。

通过下面的程序可以测试出来：

```

1 fn doit3(t1: &dyn TraitA, t2: Box) {
2     println!("{:?}", t1);
3     println!("{:?}", t2)
4 }
5 fn main() {
6     let a = AType;
7     let b = BType;
8     doit3(&a, Box::new(b));
9     println!("{:?}", a);
10    println!("{:?}", b);
11 }

```

输出:

```

13 error[E0382]: borrow of moved value: b
14     --> examples/trait_object.rs:29:22
15     |
16 26 |         let b = BType;
17     |             - move occurs because b has type BType, which does not implement th
18 27 |         doit3(&a, Box::new(b));
19     |                                   - value moved here
20 28 |         println!("{:?}", a);
21 29 |         println!("{:?}", b);
22     |                                   ^ value borrowed here after move

```

答案来自哄哄、鸠摩智和 -Hedon 

## 🔗 11 | 常见 Trait 解析：标准库中的常用 Trait 应该怎么用？

### 思考题

请举例说明 Deref 与 AsRef 的区别。

### 答案

在 Rust 中，Deref 和 AsRef 都是与引用相关的 trait，它们可以使某些类型在使用时具有类似于指针的行为，但它们的具体用途有所不同。

**Deref trait 通常与智能指针一起使用。**当我们编写 Rust 代码时，分配在堆上的值通常不是通过拷贝的方式传递或返回，而是通过使用指向它们的指针（智能指针）来传递或返回。

Deref trait 可以强制将智能指针转换成指针，从而可以使用类似于 \* 操作符这样的解引用语法访问指针指向的值。例如：

```
1 struct MyInt(i32);
2
3 impl Deref for MyInt {
4     type Target = i32;
5
6     fn deref(&self) -> &Self::Target {
7         &self.0
8     }
9 }
10
11 fn main() {
12     let my_int = MyInt(42);
13     assert_eq!(*my_int, 42);
14 }
```

在上面的代码中，我们定义了一个 `MyInt` 结构体，它包含一个 `i32` 类型的值。我们实现了 `Deref` trait，并指定了目标类型为 `i32`。我们在 `deref` 方法中返回了 `self.0`，即指向 `MyInt` 中的 `i32` 值的引用。这样，我们就可以在 `main` 函数中使用 `*my_int` 访问这个 `i32` 值。

相比之下，**AsRef trait 更加通用**。它只是将类型的引用转换为其他类型的引用。一种常见的用途是将各种字符串类型统一转换为 `&str` 类型。例如：

```
1 fn do_something<T: AsRef<str>>>(input: T) {
2     let bytes = input.as_ref().as_bytes();
3     // Do something with the bytes...
4 }
5
6 fn main() {
7     let my_str = "hello".to_string();
8     do_something(my_str);
9 }
```

在上面的代码中，我们定义了一个 `do_something` 函数，它接受任何实现了 `AsRef<str>` trait 的值。在函数内部，我们首先使用 `as_ref` 方法将输入值转换为 `&str` 类型，然后使用 `as_bytes` 方法将 `&str` 类型转换为 `&[u8]` 类型。这样，我们就可以在函数中使用字节数组

操作 `bytes` 了。在 `main` 函数中，我们传递了一个所有权字符串，它在函数中，使用 `as_ref()` 转换成了 `&str` 类型。

好了，以上就是我们第一章基础篇的思考题与答案，希望你对照着答案看一下自己的思路对不对，如果你有不同的见解，也欢迎你在评论区分享出来，我们一起讨论。我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

## 全部留言 (4)

最新 精选



thanks

2023-12-11 来自北京

03 | 所有权（下）：Rust 中借用与引用的规则是怎样的？

第一个问题，从现象看是 rust 的规则不允许这么写，但是不允许这么写的原因是为啥

作者回复：举个例子，字符串String如果在存在不可变借用的情况下，原所有权变量可写的话，比如增长这个字符串，可能会导致底层存储这个字符串的内存重新分配，这时，这个字符串在堆中的地址就变了。这个时候，基于这个String得到的&slice 会指向原来的地址，导致内存访问出错。所以rust禁止了不可变引用存在的情况下的原所有权变量的修改。



不忘初心

2023-12-06 来自四川

可以给 i8 类型做 impl 吗？基本数据类型无法实现 impl.

准确点说，应该是自身crate之外的都无法直接impl，但可以impl trait

作者回复：是的



伯阳

2023-12-06 来自北京

多刷两遍，期待老师的实战课程，能讲讲tikio么，老师

作者回复: toki有讲，很多



**Distance**

2023-12-06 来自北京

啊这 按照大纲今天不应该是 axum 嘛，我还以为这周 axum 能更完😂

作者回复: 先回顾一下，夯实下基础。哈哈

共 2 条评论 >

